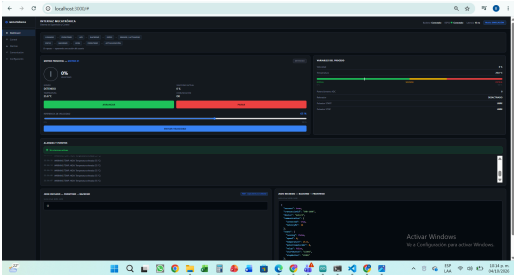
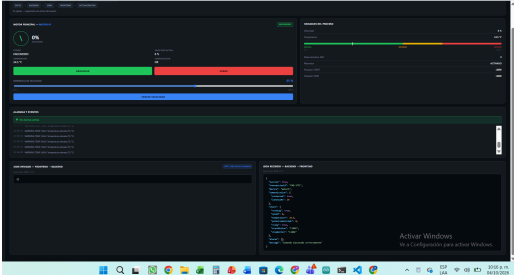
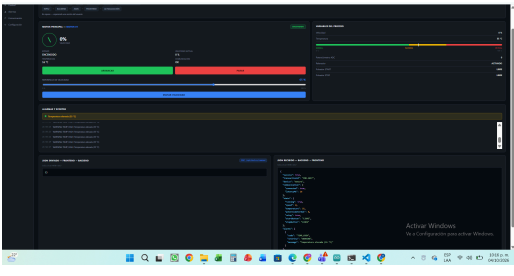
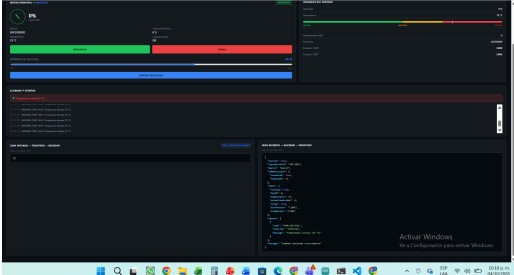
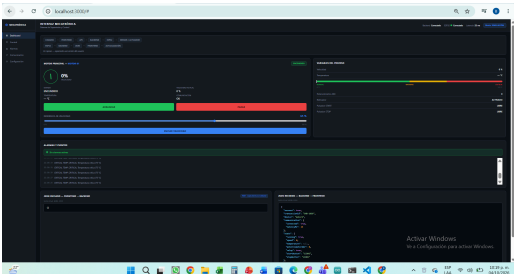
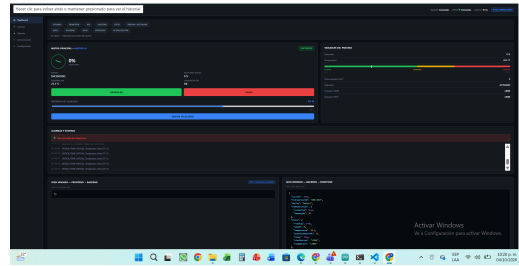


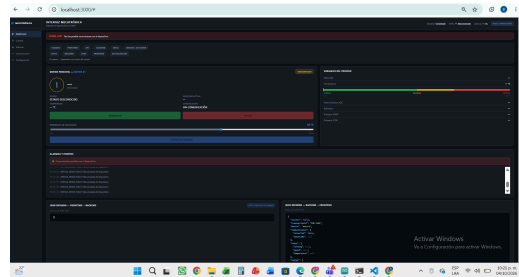
Tabla de observaciones

Paso	Escenario	Resultado observado	Evidencia
1	NORMAL	El sistema inició en estado normal. El motor estaba detenido, velocidad 0 %, temperatura de 25 °C, sin alarmas y con comunicación activa.	
2	START	Se envió el comando START y el motor pasó a estado ENCENDIDO. El relevador quedó activo y la comunicación continuó normal.	
3	TEMP_HIGH	Se forzó una temperatura de 55 °C. La interfaz mostró la condición de temperatura elevada y la alarma TEMP_HIGH con severidad WARNING.	
4	TEMP_CRITICAL	Se forzó una temperatura de 75 °C. Se generó la alarma TEMP_CRITICAL con severidad CRITICAL.	
5	SENSOR_FAULT	Se simuló una falla del sensor. La temperatura apareció como - °C, mientras que el resto de la información del sistema continuó disponible.	

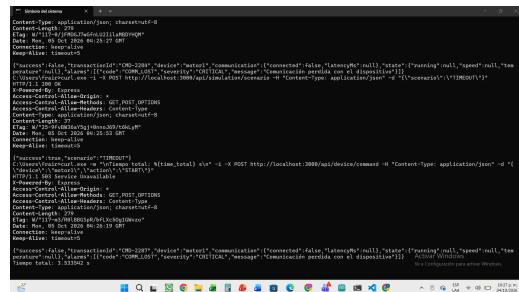
6	DEVICE_FAULT	Se simuló una falla del dispositivo. El sistema mostró DEVICE_FAULT con severidad CRITICAL.
---	--------------	---



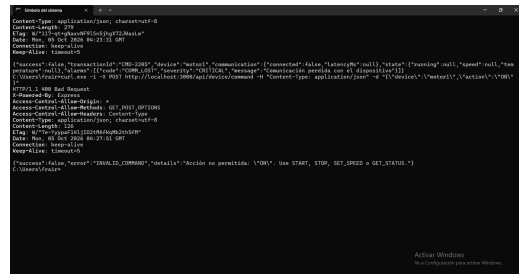
7	COMM_LOST	Se perdió la comunicación con el dispositivo. El ESP32 apareció desconectado, la latencia quedó en – y el estado del motor pasó a desconocido.
---	-----------	--



8	TIMEOUT	Se forzó el escenario TIMEOUT y se envió un comando START. El sistema tardó 3.53 segundos en responder y regresó HTTP 503. La respuesta final indicó COMM_LOST.
---	---------	--



9	Comando inválido	Se envió la acción ON, que no está permitida. El backend respondió HTTP 400 con INVALID_COMMAND y no ejecutó el comando.
---	------------------	--



Reflexión

1. ¿Cuál fue la práctica más difícil y por qué? ¿Qué parte fue menos clara?

La práctica más difícil fue la parte de diagnóstico porque se tuvieron que probar diferentes situaciones de falla y comprobar que el sistema respondiera correctamente en cada una. Lo que menos claro me pareció fue el recorrido completo de la información, desde que el usuario hace una petición en el frontend, pasa por la API y el backend, se valida, llega al dispositivo y después regresa el estado para mostrarlo nuevamente en la interfaz.

También fue importante entender que no solamente se debe mostrar un mensaje en pantalla, sino que el backend debe comprobar realmente si la operación se pudo realizar.

2. ¿Cuál es la diferencia entre lo que el usuario solicita y lo que realmente hizo el sistema?

Lo que el usuario solicita es solamente una intención, mientras que el sistema debe comprobar si realmente pudo realizar esa acción.

Por ejemplo, si el usuario presiona ARRANCAR, la interfaz puede enviar el comando START, pero eso no significa automáticamente que el motor arrancó. El backend debe recibir la respuesta del dispositivo y confirmar el estado. Si existe una falla de comunicación, el sistema debe informar que no se pudo ejecutar el comando en lugar de mostrar el motor como encendido.

Esto evita que la interfaz muestre información falsa al usuario.

3. Si se agregara un motor 2, ¿qué archivos se tendrían que modificar?

A nivel general, se tendría que modificar el frontend para agregar los controles y mostrar el estado del motor 2. En el backend se tendrían que agregar las validaciones y comandos para identificar motor2 y procesar sus estados. Finalmente, en el firmware del ESP32 se tendrían que agregar las entradas o salidas necesarias para controlar el segundo motor y enviar su estado.

También habría que actualizar las pruebas para comprobar que los comandos del motor 2 funcionen correctamente.